CHANDIGARH UNIVERSITY

Bachelor of Technology (CSE – Microsoft AI)

CASE STUDY REPORT

Case Study 4: CPU Pipelining and Performance Improvement

Subject	Computer Organization and Architecture
Submitted by	Kiranpreet Kaur
UID	25BAI10033
Section	25BAI-601
Course	B.Tech CSE (Microsoft AI)
Submitted to	Ruchika Gupta Ma'am

1. Introduction and Scenario

Whenever we run a program, the processor is really just executing one instruction after another, thousands of millions of times every second. Because of this, even a tiny saving in the time taken per instruction adds up to a very large gain overall. Instruction pipelining is one of the cleverest ways computer architects have found to get that saving without making the clock itself any faster. In this case study we look at how pipelining works, how much faster it makes a processor in the ideal situation, and why real processors never quite reach that ideal.

1.1 Scenario

A processor has to execute 10 independent instructions. Every instruction passes through five stages, and each stage takes exactly one clock cycle. For the calculations in this report we assume an ideal five-stage pipeline, which means there are no stalls, no hazards and no waiting of any kind.

1.2 Problem Statement

We have to compare the number of clock cycles needed by sequential (non-pipelined) execution with the number needed by an ideal pipelined execution, and then find out why real pipelines may perform less well than the ideal one.

1.3 Objectives

- To understand and explain the idea of instruction pipelining.
- To list the five classic stages of a RISC pipeline and say what each one does.
- To calculate the total cycles and the speedup for 10 instructions.
- To draw a pipeline timing chart showing overlapping stages.
- To study structural, data and control hazards and their effect on performance.

1.4 Assumptions

- All 10 instructions are independent of each other, so no instruction needs the result of another.
- Each of the five stages takes exactly one clock cycle, and all stages are of equal length.
- There are no stalls, hazards, cache misses or branch problems in the ideal case.

2. Case Study Questions and Answers

Q16. Explain instruction pipelining.

Instruction pipelining is a technique in which the execution of several instructions is overlapped in time. Instead of waiting for one instruction to finish completely before the next one starts, the processor breaks every instruction into small steps (called stages) and lets different instructions occupy different stages at the same moment.

The easiest way to picture this is a **laundry example**. Suppose washing, drying, ironing and folding each take 30 minutes. If one person finishes all four jobs on one load before touching the next load, four loads take 8 hours. But if the load that has been washed moves on to the dryer and the next load goes into the washing machine, all the machines stay busy, and the same four loads finish in only 5 hours. Each load still takes 2 hours from start to end, but the overall *throughput* is much better. A pipeline in a CPU works in exactly the same way, with an assembly-line idea at its core.

There are a few important points to remember about pipelining:

- **It does not reduce the time of one instruction.** A single instruction still needs all five cycles from fetch to write-back. In fact, because of the extra pipeline registers, it can be slightly slower.
- **It increases throughput.** Once the pipeline is full, one instruction finishes in every clock cycle, so the number of instructions completed per unit time goes up.
- **Pipeline registers separate the stages.** Small registers between stages hold the data and control signals of the instruction moving along, so that every stage can work on a different instruction safely.
- **The slowest stage decides the clock.** That is why stages are designed to be of nearly equal length, as in this case study.

In short, pipelining uses instruction-level parallelism: the hardware is the same, but it is used far more efficiently because no stage sits idle once the pipeline is full.

Q17. Name the five classic pipeline stages.

The classic RISC pipeline (as used in MIPS-style processors) has five stages. Every instruction goes through them in the same order, as summarised in Table 1.

To understand these better, let us follow a simple instruction such as ADD R1, R2, R3. In the IF stage it is fetched from memory. In ID, the processor works out that it is an addition and reads the values of R2 and R3. In EX, the ALU adds the two values. In MEM nothing special happens, because an addition does not touch data memory. Finally, in WB the answer is stored in R1.

Similarly, a LOAD instruction uses all five stages meaningfully, since it must calculate an address in EX, read data in MEM and place it in a register in WB. This uniform five-step structure is what makes it easy to overlap different instructions in the pipeline.

No.	Stage	Full Name	What happens in this stage
1	IF	Instruction Fetch	The instruction is read from memory using the address in the Program Counter (PC), and the PC is incremented.
2	ID	Instruction Decode / Register Fetch	The instruction is decoded, control signals are generated and the source registers are read.
3	EX	Execute / Address Calculation	The ALU performs the operation, or calculates the memory address for a load or store.
4	MEM	Memory Access	Data memory is read (load) or written (store). Other instructions simply pass through.
5	WB	Write Back	The result from the ALU or from memory is written into the destination register.

Table 1: The five classic pipeline stages

Q18. How many cycles are needed without pipelining?

Without pipelining, the processor starts an instruction only after the previous one has fully completed all five stages. So every instruction takes 5 clock cycles, and nothing overlaps.

Formula: Total cycles (sequential) = n × k

where n is the number of instructions and k is the number of stages.

Calculation: n = 10 and k = 5, so

Total cycles (sequential) = 10 × 5 = 50 cycles

So the processor needs 50 clock cycles to finish all 10 instructions one by one.

Q19. How many cycles are needed in an ideal five-stage pipeline?

In an ideal pipeline, the first instruction needs k cycles to pass through all the stages, which is the time taken to fill the pipeline. After that, one instruction finishes in every following cycle, so each of the remaining (n − 1) instructions adds only one more cycle.

Formula: Total cycles (pipelined) = k + (n − 1)

Calculation: k = 5 and n = 10, so

Total cycles (pipelined) = 5 + (10 − 1) = 5 + 9 = 14 cycles

The first instruction completes at the end of cycle 5, the second at cycle 6, the third at cycle 7 and so on, until the tenth instruction completes at cycle 14. This can be seen clearly in the timing chart on the next page.

3. Pipeline Timing Chart

The assignment asks for a chart of at least the first five instructions. Figure 1 shows the first five instructions (I1 to I5) and Figure 2 shows the complete picture for all 10 instructions. Each row is one instruction and each column is one clock cycle (C1, C2, …). The colour of a box tells us which stage the instruction is in during that cycle.

Instr.	C1	C2	C3	C4	C5	C6	C7	C8	C9
I1	IF	ID	EX	MEM	WB				
I2		IF	ID	EX	MEM	WB			
I3			IF	ID	EX	MEM	WB		
I4				IF	ID	EX	MEM	WB	
I5					IF	ID	EX	MEM	WB

Figure 1: Pipeline timing chart for the first five instructions (9 cycles)

IF ID EX MEM WB

Looking at column C5, we can see all five stages busy at the same time: I1 is in WB, I2 in MEM, I3 in EX, I4 in ID and I5 in IF. That is the moment the pipeline is completely full, and it is where we get the real benefit of overlapping. The first four cycles are the pipeline filling up, and from C5 onwards one instruction completes in every cycle.

Completion time of each instruction

The table below lists the cycle in which every instruction enters the pipeline (IF) and the cycle in which it completes (WB). It shows that after the first instruction, one instruction finishes in every single cycle.

Instruction	Starts (IF) in cycle	Completes (WB) in cycle	Cycles in pipeline
I1	1	5	5
I2	2	6	5
I3	3	7	5
I4	4	8	5
I5	5	9	5
I6	6	10	5
I7	7	11	5
I8	8	12	5
I9	9	13	5
I10	10	14	5

Table 2: Start and completion cycle of each instruction

Complete chart for all 10 instructions

Instr.	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14
I1	IF	ID	EX	MEM	WB									
I2		IF	ID	EX	MEM	WB								
I3			IF	ID	EX	MEM	WB							
I4				IF	ID	EX	MEM	WB						
I5					IF	ID	EX	MEM	WB					
I6						IF	ID	EX	MEM	WB				
I7							IF	ID	EX	MEM	WB			
I8								IF	ID	EX	MEM	WB		
I9									IF	ID	EX	MEM	WB	
I10										IF	ID	EX	MEM	WB

Figure 2: Ideal pipeline timing chart for all 10 instructions (14 cycles)

IF **ID** **EX** **MEM** **WB**

The chart has three clear phases, which are summarised in the table below.

Phase	Cycles	What is happening
Filling the pipeline	C1 – C4	More stages become busy in every cycle, and no instruction has finished yet except at the end of C5.
Full pipeline (steady state)	C5 – C10	All five stages are busy, and one instruction completes every cycle.
Draining the pipeline	C11 – C14	No new instruction enters, so the stages become idle one after another until I10 finishes in C14.

Table 3: Phases of pipeline execution

Instruction I10 enters the pipeline at C10 and leaves at C14. The 10 instructions therefore finish at cycles 5, 6, 7, 8, 9, 10, 11, 12, 13 and 14 respectively, which fully agrees with the formula $k + (n - 1) = 14$.

4. Performance Calculation

4.1 Speedup

Speedup tells us how many times faster the pipelined processor is compared with the non-pipelined one. It is the ratio of the two execution times. Since the clock cycle time is the same in our assumption, we can compare the number of cycles directly.

Speedup = (Cycles without pipelining) / (Cycles with pipelining)

Speedup = 50 / 14 ≈ 3.57

So the pipelined processor is about 3.57 times faster than the sequential one for these 10 instructions.

4.2 Why not 5 times faster?

A five-stage pipeline has a theoretical maximum speedup of 5, equal to the number of stages. We get only 3.57 because of the time lost in filling and draining the pipeline. When the number of instructions is small, this start-up time is a large part of the total. As n grows, the speedup approaches 5, as the following table shows:

Instructions (n)	Sequential 5n	Pipelined 5+(n−1)	Speedup
10	50	14	3.57
20	100	24	4.17
50	250	54	4.63
100	500	104	4.81
1000	5000	1004	4.98

Table 4: Speedup for different numbers of instructions

4.3 Efficiency and Throughput

- **Pipeline efficiency** = Speedup / k = 3.57 / 5 ≈ 0.714, that is about 71.4%. In other words, the hardware stages are usefully busy for about 71% of the total time.
- **Throughput (pipelined)** = 10 instructions / 14 cycles ≈ 0.71 instructions per cycle, compared with 10 / 50 = 0.2 instructions per cycle without pipelining.
- **Cycles saved** = 50 − 14 = 36 cycles, which is a reduction of 72% in the total execution time.

Q20. What are structural, data and control hazards?

A hazard is a situation that prevents the next instruction from executing in its proper clock cycle. When a hazard occurs the pipeline has to stall (insert idle cycles, also called bubbles), and this reduces the performance. There are three types.

(a) Structural hazard

A structural hazard happens when two instructions need the same hardware resource in the same cycle, and the hardware cannot support both. For example, if there is only one memory shared for instructions and data, a LOAD instruction in its MEM stage and a new instruction in its IF stage both

want to use memory in the same cycle. One of them has to wait. The usual solution is to add more hardware, such as separate instruction and data caches, or multiple register ports.

(b) Data hazard

A data hazard happens when an instruction depends on the result of an earlier instruction that is still in the pipeline and has not been written back yet. Consider:

```
ADD R1, R2, R3
SUB R4, R1, R5
```

SUB needs R1 in its ID stage, but ADD writes R1 only in its WB stage, which is several cycles later (read-after-write, or RAW, hazard). The common fixes are forwarding (bypassing) the result directly from EX/MEM to the next instruction, stalling the pipeline, or having the compiler reorder instructions.

(c) Control hazard

A control hazard happens because of branch and jump instructions. The processor does not know which instruction to fetch next until the branch condition is resolved, and by then it may already have fetched the wrong instructions. These must be flushed, wasting cycles. Branch prediction, delayed branches and resolving branches earlier in the pipeline are the usual solutions.

5. Why Real Pipelines Perform Less Well

The 14-cycle figure is possible only in an ideal world. In real processors the following factors reduce the speedup:

- **Hazards and stalls:** each structural, data or control hazard inserts bubbles, so the ideal rate of one instruction per cycle is not maintained.
- **Unbalanced stages:** the slowest stage sets the clock, and in practice memory access takes longer than the others.
- **Pipeline register overhead:** the registers between stages add delay to every cycle.
- **Cache misses:** a miss in IF or MEM can stall the whole pipeline for many cycles.
- **Branch mispredictions:** a wrongly predicted branch flushes several instructions that were already in the pipeline.

For example, if just 2 stall cycles occur in our 10-instruction program, the total becomes 16 cycles and the speedup falls to 50 / 16 ≈ 3.13.

The table below shows how quickly the speedup drops as stall cycles are added to our 10-instruction program (ideal total = 14 cycles).

Stall cycles	Total cycles	Cycles without pipelining	Speedup
0 (ideal)	14	50	3.57
2	16	50	3.13
4	18	50	2.78
6	20	50	2.50
10	24	50	2.08

Table 5: Effect of stall cycles on speedup

6. Conclusion

Executing 10 instructions needs 50 cycles without pipelining but only 14 cycles with an ideal five-stage pipeline, a speedup of about 3.57. The speedup rises towards 5 as more instructions are executed, but real pipelines always fall short because of hazards, unbalanced stages and other overheads. Pipelining improves throughput, not the latency of a single instruction, and it is a basic idea behind every modern processor.

The key results of this case study are collected in the table below.

Parameter	Result
Number of instructions (n)	10
Number of pipeline stages (k)	5
Cycles without pipelining (n × k)	50
Cycles with ideal pipelining (k + n − 1)	14
Speedup (50 / 14)	≈ 3.57
Pipeline efficiency (3.57 / 5)	≈ 71.4%
Maximum possible speedup (n → ∞)	5

Table 6: Summary of results

Working on this case study made it clear to us that the benefit of pipelining comes from keeping every part of the processor busy at the same time, and that a good design is one that removes the causes of stalls, such as hazards, as far as possible.

References

- Hennessy, J. L. and Patterson, D. A., Computer Architecture: A Quantitative Approach, Morgan Kaufmann.
- Patterson, D. A. and Hennessy, J. L., Computer Organization and Design: The Hardware/Software Interface, Morgan Kaufmann.
- Stallings, W., Computer Organization and Architecture: Designing for Performance, Pearson.